1. OOP **02**

如果你没有专门学习过面向对象编程，可针对以下关键词查阅资料，重点自学以下方面：

对象/属性/方法、类/构造/析构、继承/派生、重载、重写、虚方法、模板、函数对象

2. STL Vector **02A2**

a) 试仿照讲义中的示例，学习使用STL中的vector，并逐步了解它的更多接口，乃至更多的ADT；

b) 试设计若干计算任务，对vector的计算效果做一测试。

3. DSA Vector **02A3**

a) 在实例代码包中找到Vector对应的项目，编译并运行对应的测试程序；

b) 阅读主测试程序的代码，了解测试的逻辑与过程；

c) 对照测试的每个环节，阅读对应算法的代码，进一步理解测试所报告的结果。

4. Batch & Single **02B1**

在讲义中，我们选择了先实现向量区间的插入与删除接口，然后再借用它来实现单元素的插入与删除接口。

我们为何不倾向于反过来，也就是说：先实现后者，再通过反复地调用它来实现前者？

5. insert() **02B1**

a) 试确认，若向量初始的规模为n，则insert(r, A, lo, hi)的运行时间为$\mathcal{O}((hi-lo)+(n-r))$，也就是讲义中红色区间与蓝色后缀的总长度。

b) 如果颠倒向量的设计思路，首先实现单元素的插入接口insert(r,e)，然后通过反复调用该接口来实现区间插入接口insert(r, A, lo, hi)，那么后者将运行多长时间？

6. memcpy() & memmove() **02B1**

在insert(r,A,lo,hi)及remove(lo,hi)中，时间成本主要消耗于后缀的移动。

a) 示例代码是通过显式的循环来完成移动的，如果颠倒循环的方向，可能会有什么问题？何时没问题？

b) 如果改为调用memcpy()或memmove()来实现上述移动，需要注意什么方面？

7. Worst Case Of remove() **02B1**

a) 试确认：

- remove(lo,hi)消耗的时间为$\mathcal{O}(n-hi)$，仅取决于后缀$[hi,n)$的长度，hi越小复杂度越高；
- 特别地，当$hi=0$（且$lo=0$）时为最坏情况，耗时$\mathcal{O}(n)$。

b) 试确认：当$hi=0$时，其实不会删除任何元素，后缀的前移乃是无意义的原地踏步。

c) 针对这一现象，你觉得应该如何改进？

d) 就总体而言，你的改进是否值得？

8. Amortization **02B[2+3]**

讲义中证明过，若insert()操作仅限于单元素插入，则expand()的均摊时间复杂度为$\mathcal{O}(1)$。

a) 试证明：只要是按几何级数来实现expand()算法，无论选用多大的常倍数，都可保证均摊$\mathcal{O}(1)$；

b) 这里的常倍数（2、3、4甚至更大），对时间复杂度的常系数有何影响？

c) 讲义中用装填因子来衡量向量的空间利用率，上述常倍数对这方面有何影响？

d) 试查阅资料以了解，STL等实用环境中的向量是如何扩容的？如果也是几何递增方式，常倍数选作多少？

e) 若允许通过insert()批量地插入元素，expand()的均摊时间复杂度依然是$\mathcal{O}(1)$吗？

9. Amortization **02B[2+3]**

我们知道，对exapnd()的调用可分为两种情况：无需扩容时直接返回，以及真地进行了扩容。我们也知道，“exapnd()均摊时间复杂度为$\mathcal{O}(1)$”的结论，是相对于对expand()的所有调用来做均摊而得的。那么，为什么不能仅相对于后一类情况来做均摊？

10. shrink() **02B[2+3]**

a) 对照讲义的分析，阅读并理解expand()算法；

b) 进一步阅读和理解shrink()算法（尽管讲义中并未对它做更多讲解）；

c) shrink算法为何将装填因子的阈值设为1/4，而不是与expand算法相对应地取作1/2？

11. find() + dedup() **02B4**

考查本节介绍的find()及dedup()算法。

a) 试确认：dedup()算法每次调用find(_elem[k],0,k)时，目标元素_elem[k]恰好就已处在哨兵的位置。
也就是说，find()算法中哨兵的设立、复原等操作虽然无害，却毕竟多余。

b) 这类多余的操作累计耗费多少时间？会不会伤害到整体的渐近复杂度？

c) 如何消除这类多余的操作？

d) 在find(e)命中于i<k时，dedup()选择了删除元素[i]。既然此时[i]与[k]相等，可否改为删除[k]？

12. dedup() **02B4**

讲义中考查了dedup()算法中最主要的两类操作（来自find()的元素比对以及来自remove()的元素移动），并且证明：无论向量的具体组成如何，这两类操作必然共需执行$n(n+1)/2$次。

a) 如何调整该算法，使得在最好情况下只需$\mathcal{O}(n)$时间；
尽管两类操作的渐近时间成本都是常数，但就实际的成本而言，移动操作高出一到两个数量级。
因此如果有可能，我们会非常乐意通过增加比对次数，换取移动次数的减少。

b) 试在不改变减而治之的整体策略与逻辑，仍通过调用**find**()来检测重复的前提下，使得无论如何，移动操作都不会多于n次；

c) 如此调整之后，元素比较的次数会增加多少？在向量长度超过多大阈值之后，才会得不偿失？

d) 可否进一步优化dedup()算法，使得即便在最坏情况下，其复杂度也会严格地低于$\mathcal{O}(n^2)$？

13. Function Object **02B5**

a) 针对函数对象Increase，阅读示例代码包Vector项目中对应的代码，理解函数对象的形式与机制；

b) 试仿照示例代码中的形式编写更多的函数对象，实现对向量不同功能的遍历（比如减一、加倍、求和等）。

14. Bubblesort by traversal **02B5**

Bubblesort算法中的内循环，原可以看作是对向量的一趟遍历。

a) 试照此理解，通过Vector::traverse()接口来实现内循环，以替代原先的实现方式；

b) 如此实现之后，该算法的时间复杂度还是$\mathcal{O}(n^2)$吗？

15. 判等器+比较器 **02C1**

本节uniquify()的两个版本虽然都是针对有序向量，却依然使用了“!=”来做元素的比对。
也就是说，这里假定：元素的类型要么天然就支持这样的比对，要么已对这些操作符做过重载。
如果你还不了解如何通过重载以实现这样的效果，可通过查阅一些相关资料来自学。

16. 勤奋版uniquify() **02C1**

讲义指出，该版本的去重算法一旦检测出相邻的逆序对，便会迫不及待地执行remove()，由此导致总体效率过低。
进一步地我们还注意到，倘若某次remove()触发了shrink()缩容操作，则内部的数据区_elem[]便会迁往新址。那么接下来，我们是否还能够正确地访问到向量的各个元素呢？

17. 懒惰版uniquify() **02C1**

通过简洁而精巧的设计，该版本可在$\mathcal{O}(n)$时间内完成有序向量的去重操作，且空间成本仅为$\mathcal{O}(1)$。

a) 试确认：在遇到第一个重复元素之前，循环中的移动操作都是徒劳无益的——不过是自己到自己的赋值；

b) 针对这一问题，你有什么改进的思路？试实现之。

c) 这种改进是否值得？

18. Stability **02C1**

无论是勤奋版还是懒惰版的uniquify()算法，每当发现一对相等元素，总是会删掉居后者。

如此，去重的整体效果将是，在每一组互等的元素中只有最靠前者能够幸存下来。

然而作为一个稳定的算法，本来应该是保留最靠后者的。如何调整这些算法，使之具有稳定性呢?

19. Priority By Probability **02C2**

binSearch()版本A处理三个分支的次序是“小于 ~ 大于 ~ 相等”，当然，应该还有5种可能的次序。

a) 试验证：就渐进复杂度而言，它们并无实质区别。

b) 试解释：就常系数而言，它们还是有很大的区别。

c) 上述区别主要决定于哪些因素?

d) 试以32位无符号整数构成的、长度为10^6的有序序列为例，估算出上述区别。

20. Comparator **02C2**

讲义指出，binSearch()版本A的时间成本主要取决于，在while循环中S[mi]与目标e的比较操作次数；并进而由此说明，该算法的不足在于向左、向右的转向耗时不平均，大致是**1:2**的比例。

然而我们从代码可以看到，明明还有其它的比较操作及算术运算，且总共也有$\mathcal{O}(\log n)$次。

a) 为何这部分成本被“无视”了?

b) 为何左转、右转的时间成本不是**2:3**、**3:4**、**4:5**...?（提示：参看新版教材的脚注）

21. Overflow **02C2**

为了避免运算溢出，你可能会将binSearch()算法对轴点的计算方式改为“mi = (hi-lo)/2 + lo”。

a) 试确认，这样的确可以避免数位溢出;

b) 对于binSearch()适用的查找规模而言，是否有必要如此调整?

22. Duplicates **02C2**

不难发现，当有多个元素与查找目标相等时，binSearch()版本A只是“随机”地返回了其一。

试确认：此时具体返回何者，取决于查找区间的宽度，以及相等元素的数目。

23. Fibonaccian search **02C2**

对binSearch()版本A做如下推广，可得到一个通用的查找算法：

任取一个比例$0<\lambda<1$，每次都取元素$[lo\cdot(1-\lambda)+hi\cdot\lambda]$（而非固定地取中点）作为轴点[mi]

当然，从这个角度看，二分查找即可视作为$\lambda=1/2$的特例。

a) 试证明：无论λ如何取值，查找的时间复杂度都是$\mathcal{O}(\log n)$，区别仅体现在常系数;

b) 试证明：取$\lambda=\phi=0.618...$时，复杂度的常系数最小;

c) 若对称地取$\lambda=1-\phi=0.381...$，常系数会是多少?也是最小吗?

d) 若取$\lambda=\phi=0.618...$，则如何才能快捷地取出轴点[mi]?

e) 相对于在常系数方面获得的收益，是否真地值得为如此确定轴点而增加计算量?

24. Special Case **02C3**

a) binSearch()版本B为何首先需要特判“e < S[lo]”的情况?

b) 此前的版本A、此后的版本C为何都需要如此特判?

25. Empty Sequence **02C4**

a) 如果查找目标e < S[lo]，binSearch()版本C会如何处置?

b) 如果查找区间的宽度hi-lo = 0呢?

c) 在上述情况下，版本A和版本B又会如何处置?

26. binSearch(): Version C: Correctness **02C4**

本节通过建立循环不变性，证明了该算法的正确性。你还有其他证明方法吗?

27. binSearch(): Version C: Average Search Length **02C4**

本节指出，该算法总是满足：$\mathrm{ASL}_{succ} = \mathrm{ASL}_{fail} = \mathcal{O}(1.00 \cdot \log n)$

a) 试证明，这一结论对于任何$n = 2^k - 1$都成立；

b) 试证明，这一结论对任何$n > 0$都成立。

28. binSearch() **02C[2~4]**

考查二分查找的各种版本。

a) 它们都以搜索区间收缩至1甚至0作为迭代退出的条件（对应于递归版本，即相当于递归基）。
试分别验证，无论如何迭代退出时，它们最终的区间宽度的确为1或0。

b) 试证明：无论何种算法，失败情况总是比成功情况多出一种。

c) 试针对长度为$n = 2^k - 1$的向量，估算出个算法在成功情况下的平均查找长度。

d) 试针对长度为$n = 2^k - 1$的向量，估算出个算法在失败情况下的平均查找长度。

29. Loop Termination **02C[2~4]**

我们看到，binSearch()算法的三个版本，主体都是一个**while**循环。

试逐个地确认，它们的确都能终止，且在循环的出口处lo和hi的数值都正如我们所预期。

以版本C为例，你需要验证：对于任意初始时合法的lo和hi，尽管while循环的持续条件是“lo < hi”，但在出口处必有“lo == hi”，而不致出现“lo > hi”的情况。

30. intSearch() **02C5**

即便是有序序列，数据的分布也可能很畸形。试举出这样的实例，导致插值查找算法的复杂度反而过高。

31. intSearch() **02C5**

本节针对均匀分布的数据，给出了轴点的估算公式。如果换成其它的分布形式，比如高斯分布呢?

32. Interpolation/Binary/Sequential Search **02C[2~5]**

课上指出，长度不同的查找表所适用的查找算法也不同。大体来说，随着查找表长度的递减，应当以一种接力的方式来实现查找，即依次采用插值查找、二分查找、顺序查找。当然，这只是一个定性的趋势和原则，应当结合具体的应用与硬件平台来确定它们之间准确的分界线。

a) 试在你的电脑，设计实验并通过实验数据确认，何时应当从二分查找切换至顺序查找；

b) 如果有可能，可针对更多平台及应用数据进行实验，得出对应的结果。

33. bubblesort()提前终止版 **02D**

试在示例代码包中找到该算法对应的项目：

a) 适当修改测试程序并自行设计测例，了解其中**sorted**标志的演化过程，验证该算法的逻辑正确无误；

b) 对足够大的数据做实测，估计出这一改进的平均效果（比如，平均能节省多少趟扫描）。

34. bubblesort()跳跃版 **02D**

试在示例代码包中找到该算法对应的项目：

a) 适当修改测试程序并自行设计测例，了解其中last标志的演化过程，验证该算法的逻辑正确无误；

b) 对足够大的数据做实测，估计出这一改进的平均效果（比如，平均能节省多少趟扫描）。

35. mergesort() **02E**

如果选用另一常数$\lambda \neq 1/2$，总是按照$\lambda : 1-\lambda$的比例来切分子序列，归并排序的渐近复杂度将是多少?

36. mergesort() ~ merge() **02E**

该算法的实质计算无非是二路归并，讲义及示例代码中该算法的实现虽然表面上很简洁，但由于反复地申请（new）和释放（delete）空间，时间复杂度的常系数其实颇高。

a) 你认为可以如何减少这类动态内存操作，以降低复杂度的常系数？

b) 试编程实现你的办法，并做实测对比。

37. merge() **02E**

示例代码中实现的二路归并算法，只处理了**B**先耗尽的情况，那**A**先耗尽的情况呢？

38. merge() **02E**

我们注意到，即便待归并的两个序列已经完全有序，示例代码中实现的merge()仍是墨守成规，花费线性的时间以完成“归并”。

a) 试改进merge()算法，使之能在上述情况下及时返回；

b) 为此你的merge()需要多花多少时间？

c) 如此多出的时间，相对于所得收益孰大孰小？

39. In-place Mergesort In STL **02E**

STL中的归并排序对merge()算法做了调整，从而彻底避免使用更多的辅助空间。

a) 试查阅相关资料，理解其原理及实现细节；

b) 这种调整付出了什么代价？

c) 相对于空间方面的收益，是否真地值得付出这些代价？为什么？

40. Bitmap **02F1**

a) 试在示例代码包中找到该算法对应的项目，着重阅读并理解其中利用位运算（bitwise operation）实现比特位定位的原理及过程；

b) 对照02B1中的expand()算法，阅读Bitmap::expand()的代码并理解其实现原理；

c) 为保证常数时间内完成Bitmap的整体复位，02G3节介绍了Hopcroft的校验环技巧。试从算法演示包中找到对应的Excel版演示，通过实践体会和理解其原理及过程；

d) 如果只是着眼于在常数时间内整体复位，其实还有其它方法，你能想出来一种吗？

e) 相对于Hopcroft的方法，你的方法有何优势和劣势？

f) 自学C++模板类bitset，并从接口于功能的角度，与课上讲授的Bitmap做一对比。

41. Vector<int>::dedup() With A Bitmap **02F2**

在本节我们看到，若在长度为n的向量中，所有元素都是取自$[0, m)$范围内的整数，而且$m \ll n$，则借助Bitmap结构，便可以在$\mathcal{O}(n)$时间内完成去重。

a) 试验证，该算法不仅能够实现去重，同时也能完成排序；

b) 如果$n < m$甚至$n \ll m$，这个算法是否依然可行？复杂度是否依然为$\mathcal{O}(n)$？

42. Bitmap::clear() **02F3**

在Hopcroft的Bitmap中删除元素k时，若恰好F[k] = size-1，则令size--即可。若F[k] < size-1，则需要先令T[size-1]所属的校验环，与F[k]所属的校验环互换位置，从而转化为前一种情况。

a) 试对照发放的演示程序与实例代码，理解两种情况的处理过程；

b) 试验证：无论如何，如此都可以在$\mathcal{O}(1)$时间内完成删除。

43. Vector<int>::dedup() With Hopcroft's Bitmap **02F3**

a) 试说明，当整数的取值范围很大（$m > n$甚至$m \gg n$）时，借助Hopcroft的Bitmap可将向量去重算法的效率从$\mathcal{O}(n+m)$优化到$\mathcal{O}(n)$，也就是说取值范围与时间复杂度无关（尽管仍与空间复杂度相关）。

b) 作为补偿，相对于使用常规的Bitmap，这种方法有何倒退？